

torch.func.vjp#

<https://pytorch.org/docs/stable/generated/torch.func.vjp.html>

Description:

Standing for the vector-Jacobian product, returns a tuple containing the results of `func` applied to `primals` and a function that, when given cotangents, computes the reverse-mode Jacobian of `func` with respect to `primals` times cotangents. `func` (Callable) – A Python function that takes one or more arguments. Must return one or more Tensors. `primals` (Tensors) – Positional arguments to `func` that must all be Tensors. The returned function will also be computing the derivative with respect to these arguments `has_aux` (bool) – Flag indicating that `func` returns a `(output, aux)` tuple where the first element is the output of the function to be differentiated and the second element is other auxiliary objects that will not be differentiated. Default: False. Returns a `(output, vjp_fn)` tuple containing the output of `func` applied to `primals` and a function that computes the vjp of `func` with respect to all `primals` using the cotangents passed to the returned function. If `has_aux` is True, then instead returns a `(output, vjp_fn, aux)` tuple. The returned `vjp_fn` function will return a tuple of each VJP.

Function Signature

```
torch.func.vjp(func, *primals, has_aux=False)[source]#
```

Parameters

Returns

Returns:

Returns a (output, vjp_fn) tuple containing the output of func applied to primals and a function that computes the vjp of func with respect to all primals using the cotangents passed to the returned function. If has_aux is True, then instead returns a (output, vjp_fn, aux) tuple. The returned vjp_fn function will return a tuple of each VJP.

Code Examples

```
>>> x = torch.randn([5])
>>> f = lambda x: x.sin().sum()
>>> (_, vjpfunc) = torch.func.vjp(f, x)
>>> grad = vjpfunc(torch.tensor(1.0))[0]
>>> assert torch.allclose(grad, torch.func.grad(f)(x))
```

```
>>> x = torch.randn([5])
>>> f = lambda x: (x.sin(), x.cos())
>>> (_, vjpfunc) = torch.func.vjp(f, x)
>>> vjps = vjpfunc((torch.ones([5]), torch.ones([5])))
>>> assert torch.allclose(vjps[0], x.cos() + -x.sin())
```

```
>>> x = torch.randn([5])
>>> f = lambda x: {"first": x.sin(), "second": x.cos()}
>>> (_, vjpfunc) = torch.func.vjp(f, x)
>>> cotangents = {"first": torch.ones([5]), "second":
torch.ones([5])}
>>> vjps = vjpfunc(cotangents)
>>> assert torch.allclose(vjps[0], x.cos() + -x.sin())
```

```
>>> x, y = torch.randn([5, 4]), torch.randn([4, 5])
>>> (_, vjpfunc) = torch.func.vjp(torch.matmul, x, y)
>>> cotangents = torch.randn([5, 5])
>>> vjps = vjpfunc(cotangents)
>>> assert len(vjps) == 2
>>> assert torch.allclose(vjps[0], torch.matmul(cotangents,
y.transpose(0, 1)))
>>> assert torch.allclose(vjps[1], torch.matmul(x.transpose(0,
1), cotangents))
```

```
>>> x = torch.randn([5])
>>> def f(x, scale=4.):
>>>     return x * scale
>>>
>>> (_, vjpfunc) = torch.func.vjp(f, x)
>>> vjps = vjpfunc(torch.ones_like(x))
>>> assert torch.allclose(vjps[0], torch.full(x.shape, 4.0))
```

```
>>> def f(x):
>>>     with torch.no_grad():
>>>         c = x ** 2
>>>     return x - c
```

```
>>> with torch.no_grad():
>>>     vjp(f)(x)
```

Notes & Warnings

NOTE

Note Using PyTorch `torch.no_grad` together with `vpj`. Case 1: Using `torch.no_grad` inside a function:

```
>>> def f(x): >>> with torch.no_grad(): >>> c = x ** 2 >>> return x - c
```

 In this case, `vpj(f)(x)` will respect the inner `torch.no_grad`. Case 2: Using `vpj` inside `torch.no_grad` context manager:

```
>>> with torch.no_grad(): >>> vpj(f)(x)
```

 In this case, `vpj` will respect the inner `torch.no_grad`, but not the outer one. This is because `vpj` is a “function transform”: its result should not depend on the result of a context manager outside of `f`.

Detailed Documentation

Documentation

Standing for the vector-Jacobian product, returns a tuple containing the results of `func` applied to primals and a function that, when given cotangents, computes the reverse-mode Jacobian of `func` with respect to primals times cotangents.

`func` (Callable) – A Python function that takes one or more arguments. Must return one or more Tensors.

`primals` (Tensors) – Positional arguments to `func` that must all be Tensors. The returned function will also be computing the derivative with respect to these arguments

`has_aux` (bool) – Flag indicating that `func` returns a (output, aux) tuple where the first element is the output of the function to be differentiated and the second element is

other auxiliary objects that will not be differentiated. Default: False.

Returns a (output, vjp_fn) tuple containing the output of func applied to primals and a function that computes the vjp of func with respect to all primals using the cotangents passed to the returned function. If has_aux is True, then instead returns a (output, vjp_fn, aux) tuple. The returned vjp_fn function will return a tuple of each VJP.

When used in simple cases, vjp() behaves the same as grad()

However, vjp() can support functions with multiple outputs by passing in the cotangents for each of the outputs

vjp() can even support outputs being Python structs

The function returned by vjp() will compute the partials with respect to each of the primals

primals are the positional arguments for f. All kwargs use their default value

Using PyTorch torch.no_grad together with vjp. Case 1: Using torch.no_grad inside a function:

In this case, vjp(f)(x) will respect the inner torch.no_grad.

Case 2: Using vjp inside torch.no_grad context manager:

In this case, vjp will respect the inner torch.no_grad, but not the outer one. This is because vjp is a “function transform”: its result should not depend on the result of a context manager outside of f.